

**LAPORAN PROYEK**  
**SISTEM OTOMASI LAMPU DENGAN SENSOR CAHAYA**  
**BERBASIS ESP32 DAN WEB**



**UMY**  
**UNIVERSITAS**  
**MUHAMMADIYAH**  
**YOGYAKARTA**

Unggul & Islami

Disusun Oleh:

**Kelompok 4 – Sensor Cahaya**

|                               |             |
|-------------------------------|-------------|
| Ridha Novita Handayani        | 20240140022 |
| Fannandya Sutan Sakti Pratama | 20240140033 |
| Andhika Pratama               | 20240140037 |
| Ifa Asmarani Rosalba          | 20230140015 |
| Salmaa Rifhani Rayyan         | 20230140082 |
| Irza Yaumil Syahrar           | 20230140094 |

**PROGRAM STUDI S-1 TEKNOLOGI INFORMASI**  
**FAKULTAS TEKNIK**  
**UNIVERSITAS MUHAMMADIYAH YOGYAKARTA**

**2025**

## Daftar Isi

|                                          |    |
|------------------------------------------|----|
| <b>Bab 1 Pendahuluan</b>                 | 5  |
| 1.1 Latar belakang                       | 5  |
| 1.2 Tujuan                               | 5  |
| 1.3 Manfaat                              | 5  |
| 1.4 Ruang Lingkup                        | 5  |
| <b>Bab 2 Perencanaan Proyek</b>          | 7  |
| 2.1 Deskripsi Sistem                     | 7  |
| 2.2 Arsitektur Sistem                    | 7  |
| 2.3 Kebutuhan Fungsional                 | 8  |
| 2.4 Kebutuhan Non-Fungsional             | 9  |
| 2.5 Pembagian Tugas Tim                  | 9  |
| 2.6 Diagram - diagram                    | 10 |
| 2.6.1 Usecase                            | 10 |
| 2.6.2 Flowchart User                     | 11 |
| 2.6.3 Flowchart Admin                    | 11 |
| 2.6.4 Activity Diagram User              | 12 |
| 2.6.5 Activity Diagram Update Threshold  | 12 |
| 2.6.6 Activity Diagram Manual Override   | 13 |
| 2.6.7 Activity Diagram User Management   | 13 |
| 2.6.8 Sequence Diagram User              | 14 |
| 2.6.9 Sequence Diagram Admin             | 15 |
| 2.6.10 Class Diagram                     | 16 |
| 2.6.11 Entity Relationship Diagram (ERD) | 16 |
| 2.6.12 Architecture Diagram              | 17 |
| <b>Bab 3 Implementasi</b>                | 18 |
| 3.1 Struktur Folder/Proyek               | 18 |
| 3.1.1 Root Directory                     | 18 |
| 3.1.2 Backend (be/)                      | 18 |
| 3.1.3 Frontend (fe/)                     | 18 |
| 3.1.4 Database (database/)               | 18 |

|              |                                    |           |
|--------------|------------------------------------|-----------|
| 3.1.5        | IoT (iot/)                         | 19        |
| 3.2          | Implementasi Database              | 19        |
| 3.2.1        | Konfigurasi Database               | 19        |
| 3.2.2        | Skema Tabel                        | 19        |
| 3.2.3        | Relasi Antar Tabel                 | 20        |
| 3.2.4        | Contoh Migrasi                     | 20        |
| 3.2.5        | Contoh Model Sequelize             | 21        |
| 3.3          | Implementasi Backend               | 21        |
| 3.3.1        | Teknologi yang Digunakan           | 21        |
| 3.3.2        | Struktur API Endpoints             | 22        |
| 3.3.3        | Middleware                         | 22        |
| 3.3.4        | Contoh Middleware Autentikasi      | 22        |
| 3.3.5        | Contoh Controller Login            | 23        |
| 3.3.6        | Contoh Controller Dashboard Status | 24        |
| 3.4          | Implementasi Frontend              | 24        |
| 3.4.1        | Teknologi yang Digunakan           | 24        |
| 3.4.2        | Struktur Halaman                   | 25        |
| 3.4.3        | Komponen UI                        | 25        |
| 3.4.4        | API Service                        | 25        |
| 3.4.5        | Contoh Halaman Dashboard           | 26        |
| 3.5          | Implementasi IoT                   | 27        |
| 3.5.1        | Hardware yang Digunakan            | 27        |
| 3.5.2        | Alur Kerja ESP32                   | 27        |
| 3.5.3        | Kode ESP32                         | 28        |
| 3.6          | Screenshot Hasil                   | 30        |
| 3.6.1        | Tampilan User                      | 30        |
| 3.6.2        | Tampilan Admin                     | 32        |
| <b>Bab 4</b> | <b>Penutup</b>                     | <b>39</b> |

## Daftar Gambar

|                                                  |    |
|--------------------------------------------------|----|
| Gambar 1 Use Case Diagram.....                   | 10 |
| Gambar 2 Flowchart User .....                    | 11 |
| Gambar 3 Flowchart Admin.....                    | 11 |
| Gambar 4 Activity Diagram User .....             | 12 |
| Gambar 5 Activity Diagram Update Threshold ..... | 12 |
| Gambar 6 Activity Diagram Manual Override .....  | 13 |
| Gambar 7 Activity Diagram User Management .....  | 13 |
| Gambar 8 Sequence Diagram User .....             | 14 |
| Gambar 9 Sequence Diagram Admin .....            | 15 |
| Gambar 10 Class Diagram .....                    | 16 |
| Gambar 11 ERD.....                               | 16 |
| Gambar 12 Architecture Diagram .....             | 17 |
| Gambar 13 Halaman Login.....                     | 30 |
| Gambar 14 Dashboard User.....                    | 31 |
| Gambar 15 Sensor Logs User .....                 | 31 |
| Gambar 16 Statistik User .....                   | 32 |
| Gambar 17 Halaman Login.....                     | 32 |
| Gambar 18 Dashboard Admin .....                  | 33 |
| Gambar 19 Sensor Logs Admin.....                 | 33 |
| Gambar 20 Statistik Admin.....                   | 34 |
| Gambar 21 Pengaturan Light Threshold .....       | 34 |
| Gambar 22 Operating Mode Manual .....            | 35 |
| Gambar 23 Manual Lamp ON .....                   | 35 |
| Gambar 24 Manual Lamp OFF.....                   | 36 |
| Gambar 25 Operating Mode Automatic.....          | 36 |
| Gambar 26 Auto Lamp ON.....                      | 37 |
| Gambar 27 Auto Lamp OFF .....                    | 37 |
| Gambar 28 Manage User .....                      | 38 |
| Gambar 29 Perangkat IOT .....                    | 38 |

## **Bab 1 Pendahuluan**

### **1.1 Latar belakang**

Perkembangan teknologi Internet of Things (IoT) memungkinkan perangkat fisik terhubung dengan sistem informasi untuk memantau dan mengendalikan kondisi lingkungan secara real-time. Salah satu penerapannya adalah sensor cahaya yang digunakan untuk mengontrol lampu secara otomatis berdasarkan intensitas cahaya sekitar.

Dalam proyek ini dibuat Sistem Otomasi Lampu dengan Sensor Cahaya Berbasis ESP32 dan Web yang terintegrasi dengan perangkat IoT. Sistem ini memiliki dua role pengguna: Admin yang dapat mengatur nilai threshold cahaya, mengontrol lampu secara manual, serta mengelola data pengguna; dan User yang dapat memantau status lampu serta melihat riwayat data sensor melalui dashboard. Dengan sistem ini, lampu dapat menyala/mati secara otomatis serta dapat dimonitor dan dikonfigurasi melalui website secara mudah dan efisien.

### **1.2 Tujuan**

Tujuan dari proyek ini adalah:

- Membangun sistem otomasi lampu berbasis web yang terintegrasi dengan ESP32.
- Mengintegrasikan sensor cahaya (LDR) dengan sistem kontrol lampu otomatis.
- Menyajikan data sensor cahaya dalam bentuk grafik real-time dan tabel riwayat./
- Menyediakan fitur konfigurasi threshold dan kontrol lampu manual bagi admin.
- Memudahkan admin dalam mengelola data pengguna sistem.

### **1.3 Manfaat**

Manfaat dari sistem ini antara lain:

- Menghemat energi listrik dengan otomasi lampu berdasarkan intensitas cahaya.
- Memudahkan pemantauan status lampu dan kondisi cahaya secara real-time.
- Memberikan fleksibilitas bagi admin untuk mengontrol lampu secara manual jika diperlukan.
- Mempermudah pengelolaan sistem dan pengguna melalui dashboard berbasis web.

### **1.4 Ruang Lingkup**

Ruang lingkup proyek ini meliputi:

- Sistem web berbasis Next.js (Frontend) dan Express.js (Backend).
- Perangkat IoT menggunakan ESP32 dengan sensor LDR dan LED/Relay.
- Fitur login dengan dua role (Admin dan User).
- Fitur monitoring data sensor, statistik, dan grafik real-time.
- Fitur konfigurasi threshold cahaya dan kontrol lampu manual (Admin).
- Fitur pengelolaan data pengguna/CRUD (Admin).
- Sistem berfokus pada monitoring dan kontrol otomatis/manual lampu.

## Bab 2 Perencanaan Proyek

### 2.1 Deskripsi Sistem

Sistem Otomasi Lampu dengan Sensor Cahaya Berbasis ESP32 dan Web merupakan sistem yang mengintegrasikan perangkat IoT dengan aplikasi web untuk mengendalikan lampu secara otomatis berdasarkan intensitas cahaya lingkungan.

Cara Kerja Sistem:

- ESP32 membaca nilai intensitas cahaya menggunakan sensor LDR setiap 5 detik.
- ESP32 mengambil konfigurasi (threshold, mode) dari server melalui REST API.
- Jika mode Auto: lampu menyala otomatis ketika nilai LDR  $>$  threshold (gelap), dan mati ketika nilai LDR  $\leq$  threshold (terang).
- Jika mode Manual: lampu dikontrol langsung oleh Admin melalui dashboard.
- Data sensor dikirim ke server dan disimpan di database untuk monitoring dan statistik.

Pengguna Sistem:

1. Admin: Dapat memantau dashboard, mengatur threshold, mengubah mode operasi, mengontrol lampu manual, serta mengelola data pengguna (CRUD).
2. User: Dapat memantau dashboard, melihat riwayat sensor logs, dan melihat statistik (read-only).

### 2.2 Arsitektur Sistem

Sistem ini menggunakan arsitektur **Model-View-Controller (MVC)** yang dimodifikasi untuk IoT dengan teknologi sebagai berikut:

| Layer    | Teknologi                    | Fungsi                                                |
|----------|------------------------------|-------------------------------------------------------|
| Hardware | ESP32, LDR Sensor, LED/Relay | Membaca intensitas cahaya dan mengontrol lampu        |
| Backend  | Node.js + Express.js         | API Gateway untuk komunikasi antara perangkat dan web |
| Database | SQLite                       | Menyimpan data user, konfigurasi, dan log sensor      |

|          |                  |                                                     |
|----------|------------------|-----------------------------------------------------|
| Frontend | Next.js          | Dashboard interaktif untuk monitoring dan kontrol   |
| Protokol | HTTP REST (JSON) | Komunikasi data antara ESP32, Backend, dan Frontend |

Alur Komunikasi:

1. ESP32 → Backend: Mengirim data sensor (POST) dan mengambil konfigurasi (GET) setiap 5 detik.
2. Frontend → Backend: Mengambil data untuk dashboard, mengirim perubahan konfigurasi.
3. Backend → Database: Menyimpan dan mengambil data (users, system\_config, sensor\_logs).

*Diagram arsitektur sistem dapat dilihat pada Gambar 2.x (Arsitektur Diagram).*

### 2.3 Kebutuhan Fungsional

Berikut adalah daftar kebutuhan fungsional sistem berdasarkan role pengguna:

#### 1. Admin

| No | Fitur                 | Deskripsi                                                |
|----|-----------------------|----------------------------------------------------------|
| 1  | Login/Logout          | Admin dapat masuk dan keluar dari sistem                 |
| 2  | View Dashboard        | Melihat status lampu dan data sensor terbaru             |
| 3  | View Sensor Logs      | Melihat riwayat data sensor dalam bentuk tabel           |
| 4  | View Statistics       | Melihat statistik dan grafik real-time                   |
| 5  | Set Light Threshold   | Mengatur nilai batas cahaya untuk otomasi lampu          |
| 6  | Toggle Operating Mode | Mengubah mode operasi (Auto/Manual)                      |
| 7  | Manual Lamp Control   | Mengontrol lampu secara manual (ON/OFF) saat mode Manual |
| 8  | Add User              | Menambahkan pengguna baru                                |
| 9  | Delete User           | Menghapus pengguna dari sistem                           |

#### 2. User

| No | Fitur | Deskripsi |
|----|-------|-----------|
|----|-------|-----------|



|   |                  |                                                |
|---|------------------|------------------------------------------------|
| 1 | Login/Logout     | User dapat masuk dan keluar dari sistem        |
| 2 | View Dashboard   | Melihat status lampu dan data sensor terbaru   |
| 3 | View Sensor Logs | Melihat riwayat data sensor dalam bentuk tabel |
| 4 | View Statistics  | Melihat statistik dan grafik real-time         |

### 3. ESP32 Device

| No | Fitur               | Deskripsi                                           |
|----|---------------------|-----------------------------------------------------|
| 1  | Fetch Configuration | Mengambil konfigurasi (threshold, mode) dari server |
| 2  | Send Sensor Data    | Mengirim data pembacaan LDR ke server               |
| 3  | Control Lamp        | Mengontrol LED berdasarkan logika Auto/Manual       |

### 2.4 Kebutuhan Non-Fungsional

| No | Aspek                | Kebutuhan                                                                           |
|----|----------------------|-------------------------------------------------------------------------------------|
| 1  | Performa             | Sistem dapat menampilkan data sensor secara real-time dengan polling setiap 5 detik |
| 2  | Keamanan             | Sistem menggunakan autentikasi JWT untuk mengamankan akses API                      |
| 3  | Responsif            | Tampilan web dapat menyesuaikan dengan berbagai ukuran layar (desktop & mobile)     |
| 4  | Ketersediaan         | ESP32 dapat terus mengirim data selama terhubung ke jaringan WiFi                   |
| 5  | Kemudahan Penggunaan | Antarmuka web mudah dipahami dan dioperasikan oleh pengguna                         |
| 6  | Skalabilitas         | Database dapat menyimpan ribuan data log sensor                                     |

### 2.5 Pembagian Tugas Tim

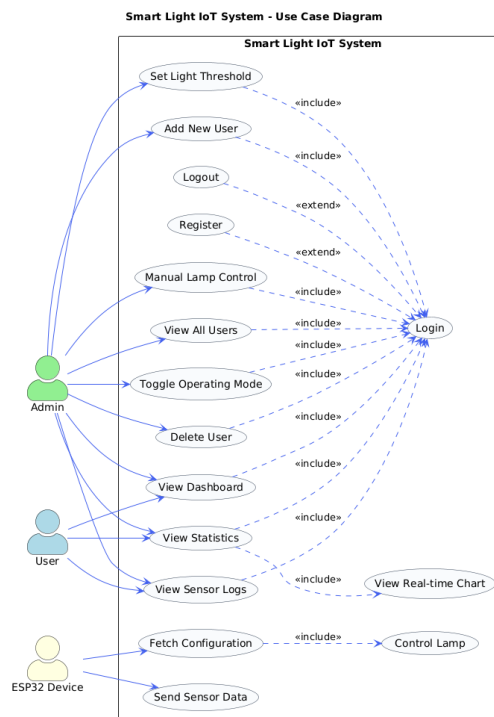
| No | Nama  | Peran             | Tugas                                                          |
|----|-------|-------------------|----------------------------------------------------------------|
| 1  | Salma | UML & Dokumentasi | Membuat diagram UML (Use Case, Sequence, Activity, Class, ERD) |

|   |              |                    |                                                            |
|---|--------------|--------------------|------------------------------------------------------------|
| 2 | <b>Dhika</b> | Backend Developer  | Membangun REST API menggunakan Express.js                  |
| 3 | <b>Tama</b>  | Hardware/Arduino   | Memprogram ESP32 untuk membaca sensor dan mengontrol lampu |
| 4 | <b>Ridha</b> | Database           | Merancang dan membangun skema database SQLite              |
| 5 | <b>Irza</b>  | Frontend Developer | Membangun dashboard web menggunakan Next.js                |
| 6 | <b>Rani</b>  | Laporan            | Menyusun dan mengompilasi laporan akhir proyek             |

## 2.6 Diagram - diagram

### 2.6.1 Usecase

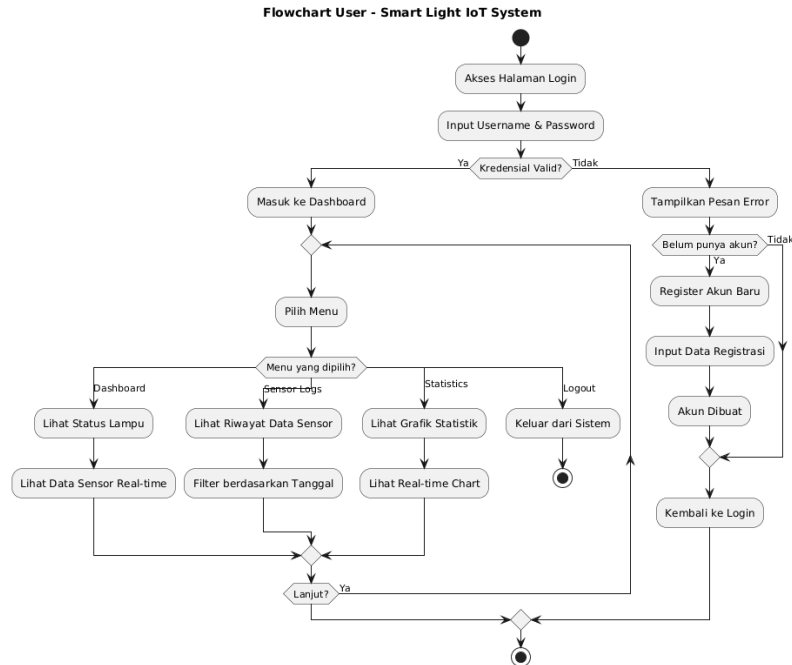
Use Case Diagram menggambarkan interaksi antara aktor (Admin, User, ESP32 Device) dengan sistem, meliputi fitur login, monitoring, konfigurasi lampu, dan pengelolaan user.



Gambar 1 Use Case Diagram

## 2.6.2 Flowchart User

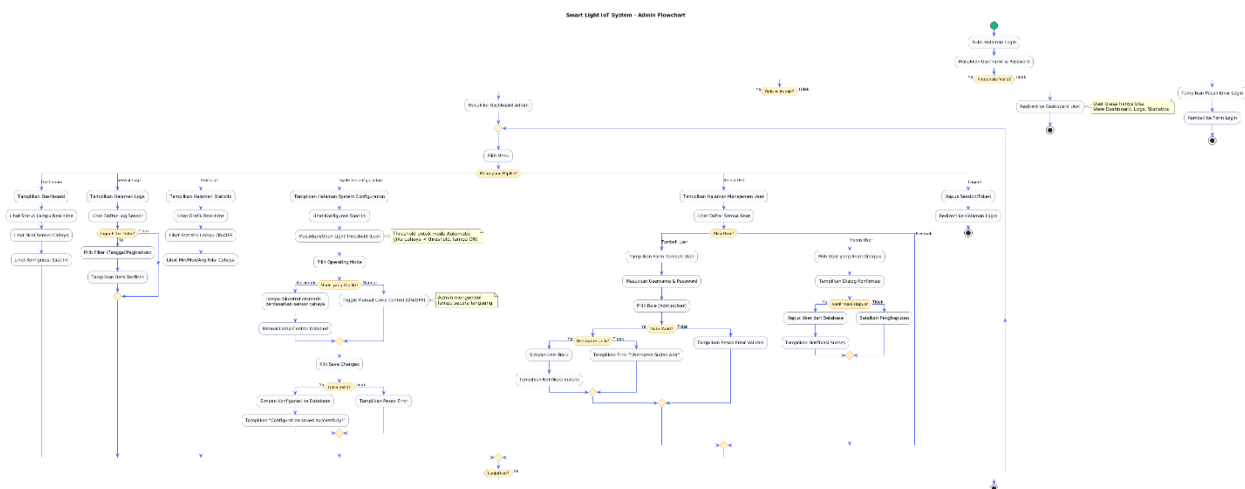
Flowchart User menggambarkan alur aktivitas pengguna dari login hingga melihat dashboard, sensor logs, dan statistik.



Gambar 2 Flowchart User

## 2.6.3 Flowchart Admin

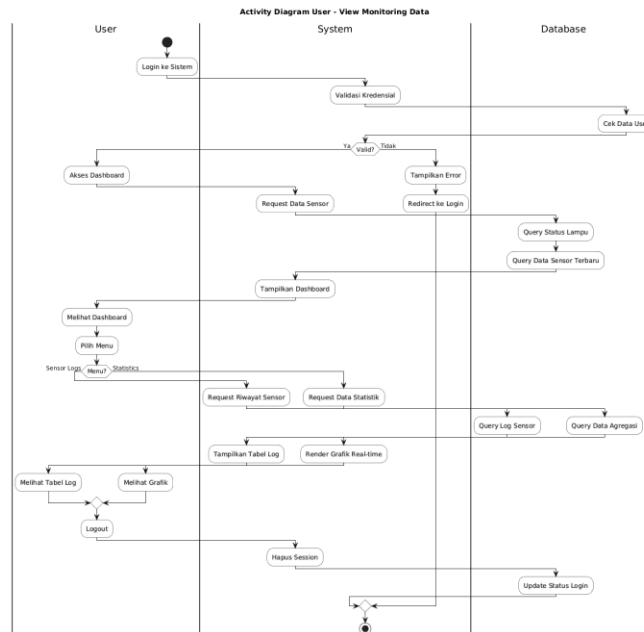
Flowchart Admin menggambarkan alur kerja admin dari login hingga monitoring, konfigurasi lampu, dan pengelolaan user (CRUD).



Gambar 3 Flowchart Admin

## 2.6.4 Activity Diagram User

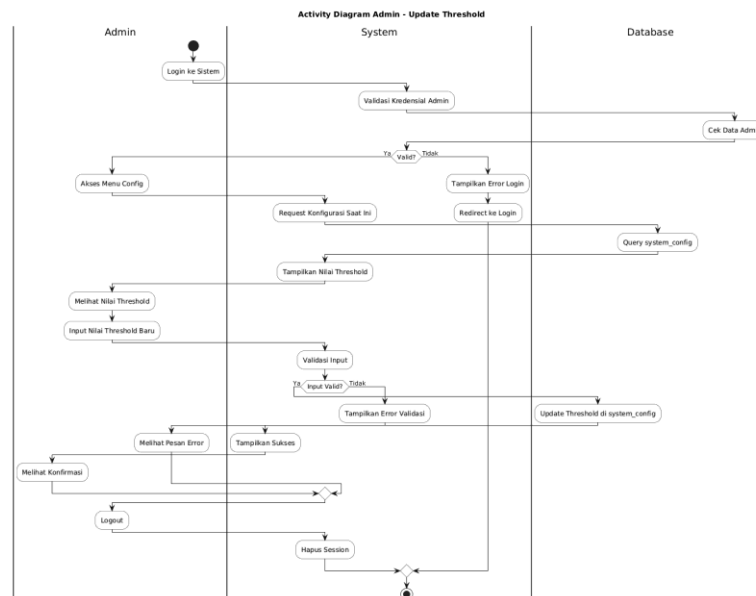
Activity Diagram User menggambarkan alur aktivitas pengguna dengan swimlane User, System, dan Database, mulai dari login hingga melihat dashboard, sensor logs, statistik, dan logout.



Gambar 4 Activity Diagram User

## 2.6.5 Activity Diagram Update Threshold

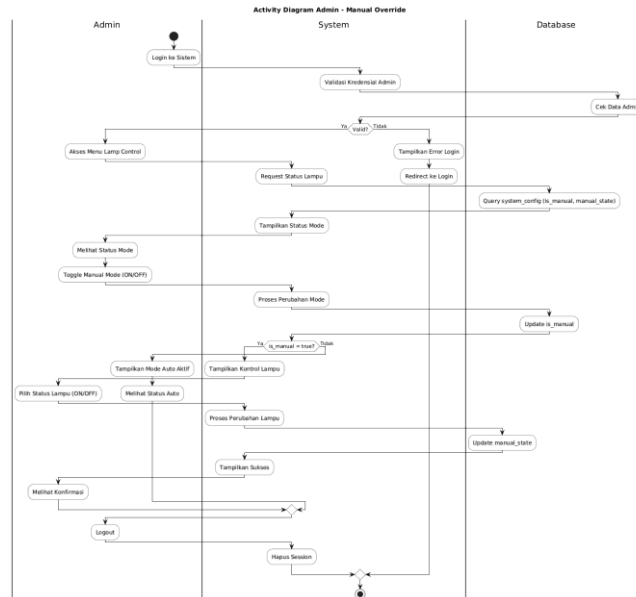
Activity Diagram Update Threshold menggambarkan alur admin mengubah nilai threshold, dari login hingga menyimpan perubahan ke database.



Gambar 5 Activity Diagram Update Threshold

## 2.6.6 Activity Diagram Manual Override

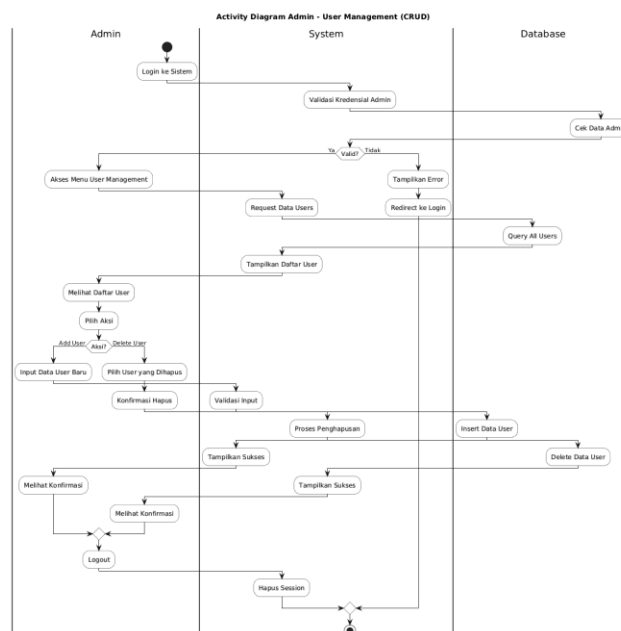
Activity Diagram Manual Override menggambarkan alur admin mengontrol lampu secara manual, dari login hingga mengaktifkan mode manual dan mengubah status lampu.



Gambar 6 Activity Diagram Manual Override

## 2.6.7 Activity Diagram User Management

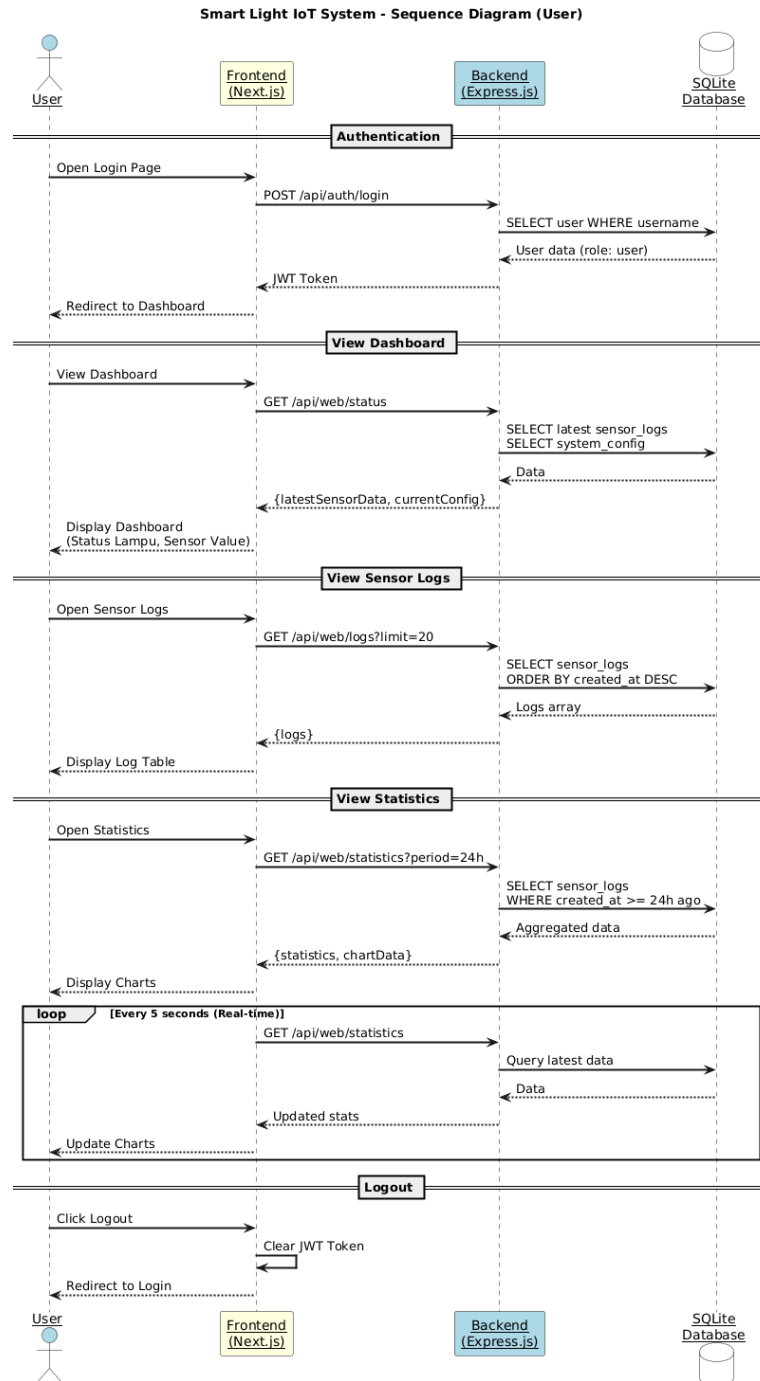
Activity Diagram User Management menggambarkan alur admin mengelola data pengguna (CRUD), dari login hingga tambah, edit, atau hapus user.



Gambar 7 Activity Diagram User Management

## 2.6.8 Sequence Diagram User

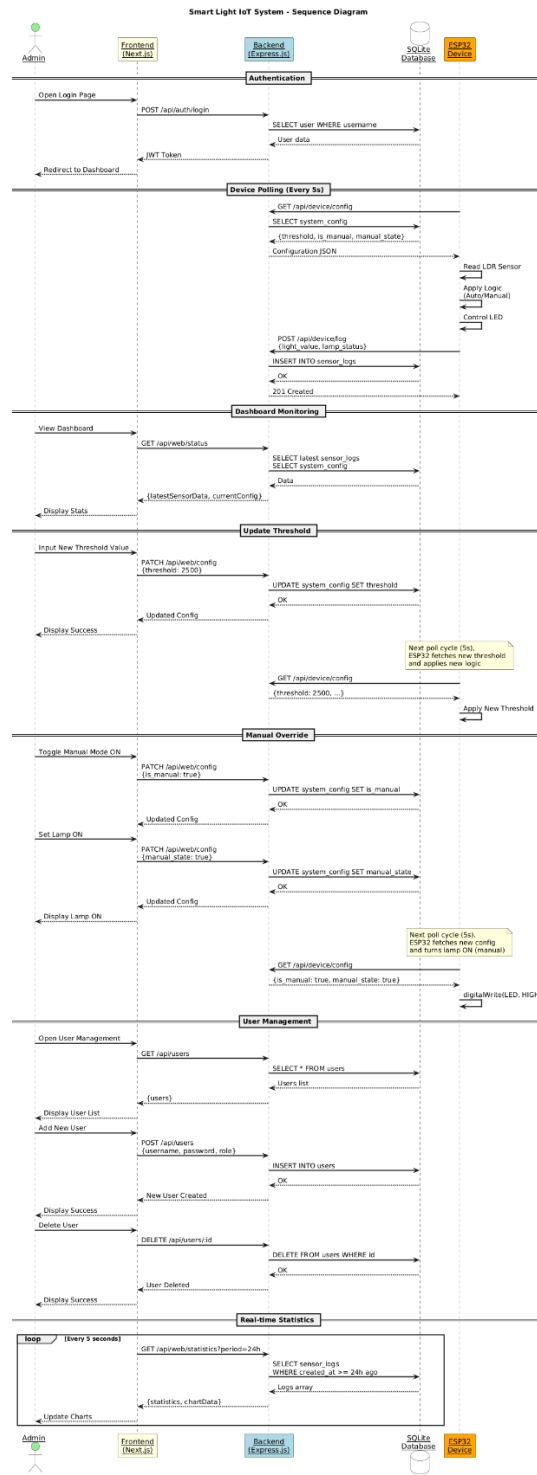
Sequence Diagram User menggambarkan alur komunikasi antara User, Frontend, Backend, dan Database saat melakukan login, melihat dashboard, sensor logs, dan statistik.



Gambar 8 Sequence Diagram User

## 2.6.9 Sequence Diagram Admin

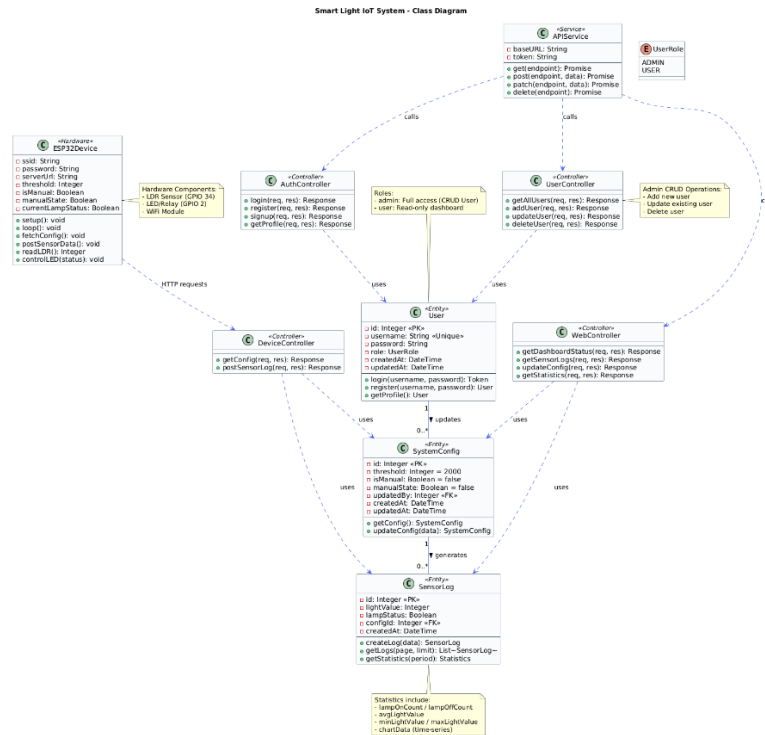
Sequence Diagram Admin menggambarkan alur komunikasi saat admin melakukan update threshold, manual override, dan user management, termasuk interaksi dengan ESP32 Device.



Gambar 9 Sequence Diagram Admin

## 2.6.10 Class Diagram

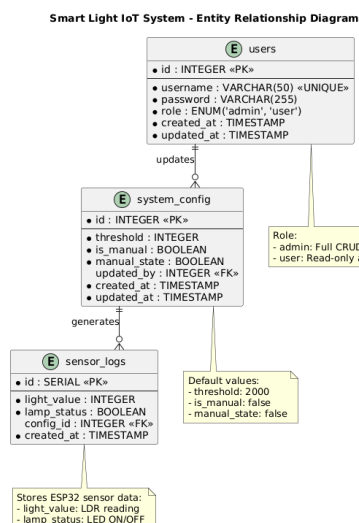
Class Diagram menggambarkan struktur kelas sistem meliputi Entity (User, SystemConfig, SensorLog), Controller, Service, dan ESP32Device beserta relasinya.



Gambar 10 Class Diagram

## 2.6.11 Entity Relationship Diagram (ERD)

ERD menggambarkan struktur database dengan tiga tabel utama: users, system\_config, dan sensor\_logs beserta relasi antar tabel.

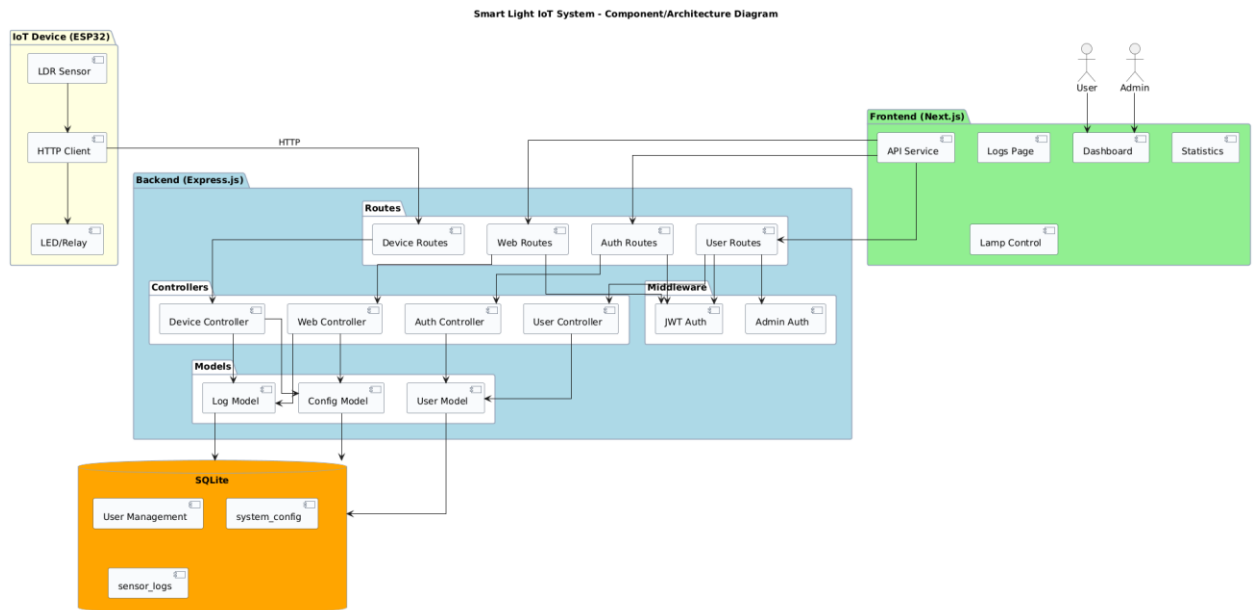


Gambar 11 ERD



## 2.6.12 Architecture Diagram

Component Diagram menggambarkan arsitektur sistem secara keseluruhan, meliputi layer Hardware (ESP32), Backend (Express.js), Database (SQLite), dan Frontend (Next.js).



Gambar 12 Architecture Diagram

## Bab 3 Implementasi

### 3.1 Struktur Folder/Proyek

#### 3.1.1 Root Directory

| Folder/File | Keterangan                            |
|-------------|---------------------------------------|
| be/         | Backend API (Node.js + Express)       |
| fe/         | Frontend Web (Next.js)                |
| database/   | Konfigurasi dan skema database SQLite |
| iot/        | Kode mikrokontroler ESP32             |
| uml/        | Diagram-diagram UML proyek            |

#### 3.1.2 Backend (be/)

| Folder/File  | Keterangan                       |
|--------------|----------------------------------|
| config/      | Konfigurasi database             |
| controllers/ | Logic handler tiap endpoint API  |
| middleware/  | Middleware (auth, validasi, dll) |
| migrations/  | File migrasi struktur database   |
| models/      | Model Sequelize (ORM)            |
| routes/      | Definisi routing API             |
| seeders/     | Data awal untuk seeding database |
| utils/       | Fungsi helper/utilitas           |
| server.js    | Entry point server Express       |

#### 3.1.3 Frontend (fe/)

| Folder/File     | Keterangan                         |
|-----------------|------------------------------------|
| src/app/        | Halaman-halaman aplikasi (routing) |
| src/components/ | Komponen UI reusable               |
| src/context/    | State management (React Context)   |
| src/services/   | Service untuk API call             |
| src/lib/        | Library/utility functions          |
| public/         | Asset statis (gambar, icon)        |

#### 3.1.4 Database (database/)

| Folder/File | Keterangan                 |
|-------------|----------------------------|
| config/     | Konfigurasi koneksi SQLite |
| migrations/ | File migrasi tabel         |

|          |                      |
|----------|----------------------|
| models/  | Definisi model/tabel |
| seeders/ | Data seed awal       |

### 3.1.5 IoT (iot/)

| Folder/File     | Keterangan                                                        |
|-----------------|-------------------------------------------------------------------|
| smart_light.ino | Kode Arduino untuk ESP32 (baca sensor, kontrol lampu, kirim data) |

## 3.2 Implementasi Database

### 3.2.1 Konfigurasi Database

Proyek ini menggunakan SQLite sebagai database dengan konfigurasi berikut:

```
{
  "development": {
    "dialect": "sqlite",
    "storage": "../database/database.db"
  }
}
```

### 3.2.2 Skema Tabel

| Tabel         | Deskripsi                                             |
|---------------|-------------------------------------------------------|
| Users         | Menyimpan data pengguna (admin & user)                |
| SensorLogs    | Menyimpan log data sensor cahaya                      |
| SystemConfigs | Menyimpan konfigurasi sistem (threshold, mode manual) |

Tabel Users

| Field     | Tipe Data             | Keterangan                     |
|-----------|-----------------------|--------------------------------|
| id        | INTEGER               | Primary Key, Auto Increment    |
| username  | STRING                | Username unik                  |
| password  | STRING                | Password (hashed)              |
| role      | ENUM('admin', 'user') | Role pengguna, default: 'user' |
| createdAt | DATE                  | Waktu pembuatan                |
| updatedAt | DATE                  | Waktu update terakhir          |

Tabel SensorLogs

| Field      | Tipe Data | Keterangan                   |
|------------|-----------|------------------------------|
| id         | INTEGER   | Primary Key, Auto Increment  |
| lightValue | INTEGER   | Nilai sensor cahaya (LDR)    |
| lampStatus | BOOLEAN   | Status lampu (ON/OFF)        |
| configId   | INTEGER   | Foreign Key ke SystemConfigs |

|           |      |                      |
|-----------|------|----------------------|
| createdAt | DATE | Waktu pencatatan log |
|-----------|------|----------------------|

Tabel SystemConfigs

| Field      | Tipe Data | Keterangan                        |
|------------|-----------|-----------------------------------|
| id         | INTEGER   | Primary Key, Auto Increment       |
| threshold  | INTEGER   | Batas nilai sensor, default: 2000 |
| manualMode | BOOLEAN   | Mode manual, default: false       |
| updatedAt  | INTEGER   | Foreign Key ke Users              |
| createdAt  | DATE      | Waktu pembuatan                   |
| updatedAt  | DATE      | Waktu update terakhir             |

### 3.2.3 Relasi Antar Tabel

- Users → SystemConfigs: Satu user dapat mengupdate banyak konfigurasi
- SystemConfigs → SensorLogs: Satu konfigurasi dapat memiliki banyak log sensor

### 3.2.4 Contoh Migrasi

File: 20260104192223-create-user.js

```
'use strict';
module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.createTable('Users', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      username: {
        type: Sequelize.STRING,
        allowNull: false,
        unique: true
      },
      password: {
        type: Sequelize.STRING,
        allowNull: false
      },
      role: {
        type: Sequelize.ENUM('admin', 'user'),
        defaultValue: 'user'
      },
      createdAt: { allowNull: false, type: Sequelize.DATE },
      updatedAt: { allowNull: false, type: Sequelize.DATE }
    });
  }
};
```

```

    });
  },
  down: async (queryInterface) => {
    await queryInterface.dropTable('Users');
  }
};

```

### 3.2.5 Contoh Model Sequelize

File: models/user.js

```

'use strict';
const { Model } = require('sequelize');
module.exports = (sequelize, DataTypes) => {
  class User extends Model {
    static associate(models) {
      User.hasMany(models.SystemConfig, { foreignKey: 'updatedBy' });
    }
  }
  User.init({
    username: {
      type: DataTypes.STRING,
      allowNull: false,
      unique: true
    },
    password: {
      type: DataTypes.STRING,
      allowNull: false
    },
    role: {
      type: DataTypes.ENUM('admin', 'user'),
      defaultValue: 'user'
    }
  }, {
    sequelize,
    modelName: 'User',
    tableName: 'Users'
  });
  return User;
};

```

## 3.3 Implementasi Backend

### 3.3.1 Teknologi yang Digunakan

| Teknologi  | Fungsi                         |
|------------|--------------------------------|
| Node.js    | Runtime JavaScript server-side |
| Express.js | Framework web untuk API        |
| Sequelize  | ORM untuk SQLite               |

|        |                   |
|--------|-------------------|
| JWT    | Autentikasi token |
| bcrypt | Hashing password  |

### 3.3.2 Struktur API Endpoints

#### Auth Routes (/api/auth/)

| Method | Endpoint  | Deskripsi                      | Akses   |
|--------|-----------|--------------------------------|---------|
| POST   | /login    | Login user, dapatkan token JWT | Public  |
| POST   | /signup   | Register user baru             | Public  |
| POST   | /register | Register user oleh admin       | Admin   |
| GET    | /profile  | Dapatkan profil user           | Private |

#### Device Routes (/api/device/)

| Method | Endpoint | Deskripsi                     | Akses  |
|--------|----------|-------------------------------|--------|
| GET    | /config  | Ambil konfigurasi untuk ESP32 | Public |
| POST   | /log     | Kirim data sensor dari ESP32  | Public |

#### Web Routes (/api/web)

| Method | Endpoint    | Deskripsi                     | Akses   |
|--------|-------------|-------------------------------|---------|
| GET    | /status     | Ambil status dashboard        | Private |
| GET    | /logs       | Ambil log sensor (pagination) | Private |
| PATCH  | /config     | Update konfigurasi sistem     | Admin   |
| GET    | /statistics | Ambil statistik sensor        | Private |

#### User Routes (/api/users)

| Method | Endpoint | Deskripsi         | Akses |
|--------|----------|-------------------|-------|
| GET    | /        | Ambil semua users | Admin |
| PATCH  | /:id     | Update user       | Admin |
| DELETE | /:id     | Hapus user        | Admin |

### 3.3.3 Middleware

| Middleware              | Fungsi                 |
|-------------------------|------------------------|
| authMiddleware.js       | Verifikasi JWT token   |
| roleMiddleware.js       | Cek role admin         |
| validationMiddleware.js | Validasi input request |
| errorHandler.js         | Handle error global    |

### 3.3.4 Contoh Middleware Autentikasi

File: middleware/authMiddleware.js

```
const { verifyToken } = require('../utils/jwtUtils');
const { errorResponse } = require('../utils/responseFormatter');
```

```

const authenticate = (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return ErrorResponse(res, 'No token provided', 401);
    }

    const token = authHeader.substring(7); // Remove 'Bearer ' prefix
    const decoded = verifyToken(token);

    req.user = decoded; // Attach user data to request
    next();
  } catch (error) {
    return ErrorResponse(res, 'Invalid or expired token', 401);
  }
};
module.exports = { authenticate };

```

### 3.3.5 Contoh Controller Login

File: controllers/authController.js

```

const login = async (req, res, next) => {
  try {
    const { username, password } = req.body;
    // Find user by username
    const user = await db.User.findOne({ where: { username } });
    if (!user) {
      return ErrorResponse(res, 'Invalid credentials', 401);
    }
    // Compare password
    const isPasswordValid = await comparePassword(password, user.password);
    if (!isPasswordValid) {
      return ErrorResponse(res, 'Invalid credentials', 401);
    }
    // Generate JWT token
    const token = generateToken({
      id: user.id,
      username: user.username,
      role: user.role
    });
    return successResponse(res, {
      token,
      user: {
        id: user.id,
        username: user.username,
        role: user.role
      }
    }, 'Login successful', 200);
  }
};

```

```

    } catch (error) {
      next(error);
    }
  };
};

```

### 3.3.6 Contoh Controller Dashboard Status

File: controllers/webController.js

```

const getDashboardStatus = async (req, res, next) => {
  try {
    // Get latest sensor log
    const latestLog = await db.SensorLog.findOne({
      order: [['createdAt', 'DESC']],
      attributes: ['id', 'lightValue', 'lampStatus', 'createdAt']
    });
    // Get current config
    const config = await db.SystemConfig.findOne({
      order: [['updatedAt', 'DESC']],
      attributes: ['id', 'threshold', 'manualMode', 'lampStatus',
'updatedAt'],
      include: [{
        model: db.User,
        attributes: ['username'],
        required: false
      }]
    });
    return successResponse(res, {
      latestSensorData: latestLog || null,
      currentConfig: config || { threshold: 2000, manualMode: false,
lampStatus: false }
    }, 'Dashboard status retrieved successfully');
  } catch (error) {
    next(error);
  }
};

```

## 3.4 Implementasi Frontend

### 3.4.1 Teknologi yang Digunakan

| Teknologi    | Fungsi                                 |
|--------------|----------------------------------------|
| Next.js      | Framework React dengan routing         |
| React Hooks  | State management (useState, useEffect) |
| TailwindCSS  | Styling komponen                       |
| Lucide Icons | Icon library                           |



### 3.4.2 Struktur Halaman

| Halaman    | Path        | Deskripsi                            |
|------------|-------------|--------------------------------------|
| Login      | /login      | Halaman login user                   |
| Register   | /register   | Halaman registrasi user baru         |
| Dashboard  | /dashboard  | Tampilan utama status sensor & lampu |
| Config     | /config     | Pengaturan threshold & mode (Admin)  |
| Logs       | /logs       | Riwayat log sensor                   |
| Statistics | /statistics | Statistik penggunaan lampu           |
| Users      | /users      | Manajemen user (Admin)               |

### 3.4.3 Komponen UI

| Komponen  | Fungsi                      |
|-----------|-----------------------------|
| Button.js | Tombol dengan loading state |
| Card.js   | Container card untuk konten |
| Input.js  | Input field form            |

### 3.4.4 API Service

File: services/api.js

```
const API_URL = 'http://localhost:3000/api';
const getAuthHeaders = () => {
  if (typeof window !== 'undefined') {
    const token = localStorage.getItem('token');
    return token ? { 'Authorization': `Bearer ${token}` } : {};
  }
  return {};
};
export const api = {
  async get(endpoint) {
    const res = await fetch(`${API_URL}${endpoint}`, {
      headers: { ...getAuthHeaders() }
    });
    if (!res.ok) throw await res.json();
    return res.json();
  },
  async post(endpoint, body) {
    const res = await fetch(`${API_URL}${endpoint}`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        ...getAuthHeaders()
      },
      body: JSON.stringify(body)
    });
    if (!res.ok) throw await res.json();
  }
};
```

```

        return res.json();
    },

    async patch(endpoint, body) {
        const res = await fetch(`${API_URL}${endpoint}`, {
            method: 'PATCH',
            headers: {
                'Content-Type': 'application/json',
                ...getAuthHeaders()
            },
            body: JSON.stringify(body)
        });
        if (!res.ok) throw await res.json();
        return res.json();
    }
};

```

### 3.4.5 Contoh Halaman Dashboard

File: app/(dashboard)/dashboard/page.js

```

'use client';
import { useState, useEffect } from 'react';
import { Card } from '@components/ui/Card';
import { api } from '@services/api';
import { Sun, Lightbulb, Settings2 } from 'lucide-react';
export default function Dashboard() {
    const [data, setData] = useState({
        logs: { value: 0 },
        status: 'OFF',
        config: { threshold: 0, mode: 'manual' }
    });
    const [loading, setLoading] = useState(true);
    const fetchData = async () => {
        try {
            const result = await api.get('/web/status');
            if (result.data) {
                const { latestSensorData, currentConfig } = result.data;
                setData({
                    logs: { value: latestSensorData?.lightValue || 0 },
                    status: latestSensorData?.lampStatus ? 'ON' : 'OFF',
                    config: {
                        threshold: currentConfig?.threshold || 0,
                        mode: currentConfig?.manualMode ? 'manual' : 'automatic'
                    }
                });
            }
        } catch (error) {
            console.error('Failed to fetch dashboard data', error);
        }
    };
}

```

```

    } finally {
      setLoading(false);
    }
  };
  useEffect(() => {
    fetchData();
    const interval = setInterval(fetchData, 10000); // Auto refresh 10s
    return () => clearInterval(interval);
  }, []);
  return (
    <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
      { /* Light Intensity Card */ }
      <Card>
        <Sun className="h-6 w-6" />
        <h3>Light Intensity</h3>
        <div className="text-4xl font-bold">{data.logs?.value || 0}</div>
      </Card>
      { /* Lamp Status Card */ }
      <Card>
        <Lightbulb className="h-6 w-6" />
        <h3>Lamp Status</h3>
        <div className="text-4xl font-bold">{data.status}</div>
      </Card>
      { /* Threshold Card */ }
      <Card>
        <Settings2 className="h-6 w-6" />
        <h3>Threshold</h3>
        <div className="text-4xl font-bold">{data.config?.threshold ||
0}</div>
      </Card>
    </div>
  );
}

```

### 3.5 Implementasi IoT

#### 3.5.1 Hardware yang Digunakan

| Komponen    | Fungsi                                   |
|-------------|------------------------------------------|
| ESP32       | Mikrokontroler utama dengan WiFi         |
| LDR Sensor  | Sensor cahaya (Light Dependent Resistor) |
| LED / Relay | Aktuator lampu                           |

#### 3.5.2 Alur Kerja ESP32

- Koneksi WiFi
- GET /api/device/config → Ambil threshold & mode

- Baca nilai sensor LDR
- Tentukan status lampu:
  - Mode Auto:  $\text{lightValue} < \text{threshold} \rightarrow \text{Lampu ON}$
  - Mode Manual: Ikuti `manualLampStatus` dari server
- Aktuasi LED/Relay
- POST `/api/device/log`  $\rightarrow$  Kirim data ke server
- Ulangi setiap 2 detik

### 3.5.3 Kode ESP32

File: `iot/smart_light.ino`

#### Konfigurasi & Setup

```
#include <ArduinoJson.h>
#include <HttpClient.h>
#include <WiFi.h>
// CONFIGURATION
const char *ssid = "pococu";
const char *password = "12345678";
const char *serverUrl = "http://10.156.79.246:3000/api/device";
// PINS
const int LDR_PIN = 34;    // Analog input untuk LDR
const int LED_PIN = 2;     // LED internal / Relay
const int BUTTON_PIN = 0;  // Tombol boot (opsional)
// VARIABLES
int threshold = 2000;
bool manualMode = false;
bool manualLampStatus = false;
bool currentLampStatus = false;
unsigned long timerDelay = 2000; // Kirim data tiap 2 detik
void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    pinMode(LDR_PIN, INPUT);
    // Connect to WiFi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("Connected to WiFi");
}
```

#### Loop Utama (Baca Sensor & Kontrol Lampu)

```
void loop() {
    if ((millis() - lastTime) > timerDelay) {
        if (WiFi.status() == WL_CONNECTED) {
```

```

// 1. GET CONFIG dari server
fetchConfig();
// 2. BACA SENSOR
int lightValue = analogRead(LDR_PIN);
// 3. TENTUKAN STATUS LAMPU
if (!manualMode) {
    // MODE OTOMATIS
    if (lightValue < threshold) {
        currentLampStatus = true; // Gelap → Lampu ON
    } else {
        currentLampStatus = false; // Terang → Lampu OFF
    }
} else {
    // MODE MANUAL
    currentLampStatus = manualLampStatus;
}
// 4. AKTUASI LED (ESP32 LED active-LOW)
digitalWrite(LED_PIN, currentLampStatus ? LOW : HIGH);
// 5. KIRIM LOG KE SERVER
postSensorData(lightValue, currentLampStatus);
}
lastTime = millis();
}
}

```

### Fungsi Ambil Konfigurasi dari Server

```

void fetchConfig() {
    HTTPClient http;
    String url = String(serverUrl) + "/config";
    http.begin(url.c_str());
    int httpResponseCode = http.GET();
    if (httpResponseCode > 0) {
        String payload = http.getString();
        DynamicJsonDocument doc(1024);
        deserializeJson(doc, payload);
        threshold = doc["data"]["threshold"].as<int>();
        manualMode = doc["data"]["manualMode"].as<bool>();
        manualLampStatus = doc["data"]["lampStatus"].as<bool>();
        Serial.printf("Config -> Threshold: %d, Manual: %s\n",
            threshold, manualMode ? "TRUE" : "FALSE");
    }
    http.end();
}

```

### Fungsi Kirim Data Sensor ke Server

```

void postSensorData(int lightValue, bool lampStatus) {
    HTTPClient http;
    String url = String(serverUrl) + "/log";

```

```

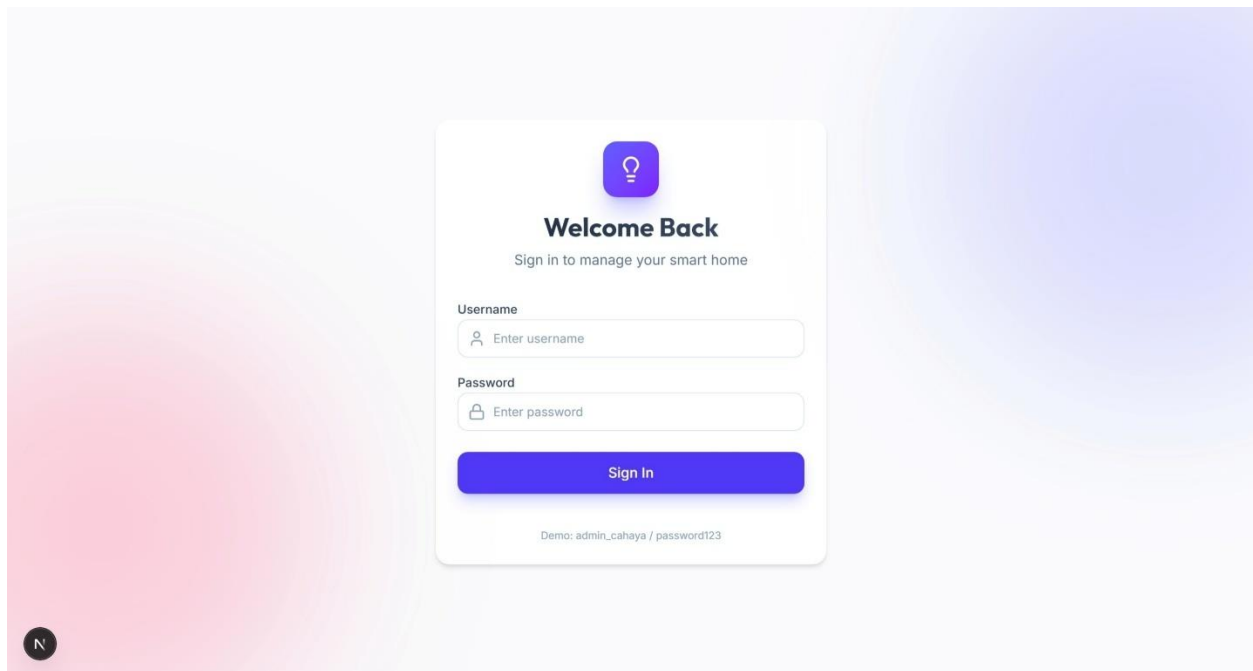
http.begin(url.c_str());
http.addHeader("Content-Type", "application/json");
// JSON Body
String jsonBody = "{\"lightValue\":\"" + String(lightValue) +
                  "\",\"lampStatus\":\"" + (lampStatus ? "true" : "false") + "\"}";
int httpResponseCode = http.POST(jsonBody);
if (httpResponseCode > 0) {
    Serial.println("Data Posted. Response: " + String(httpResponseCode));
} else {
    Serial.print("Error sending POST: ");
    Serial.println(httpResponseCode);
}
http.end();
}

```

### 3.6 Screenshot Hasil

#### 3.6.1 Tampilan User

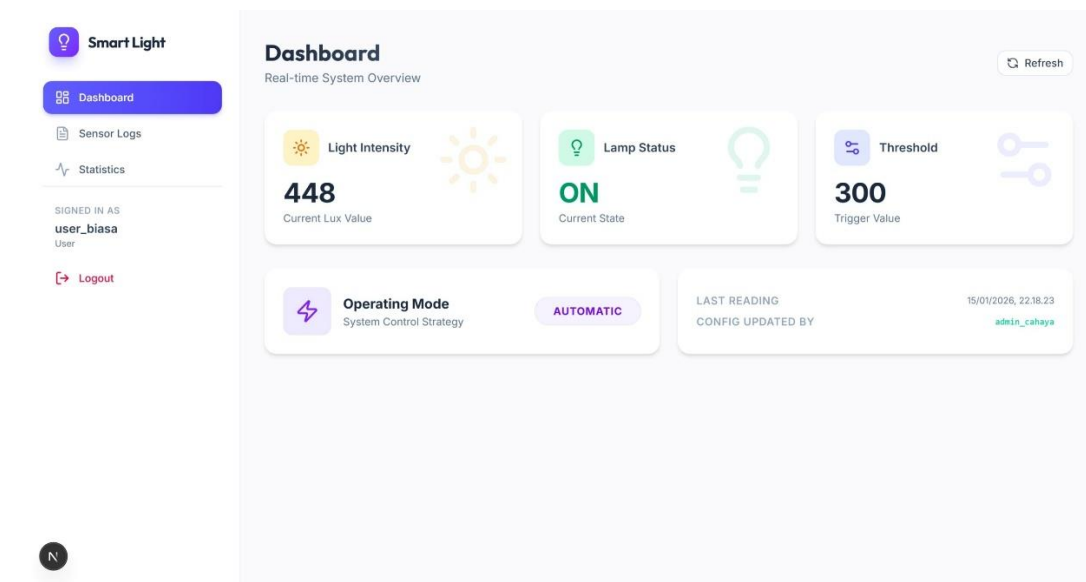
##### 1. Halaman Login



*Gambar 13 Halaman Login*

Menampilkan halaman login sistem. User memasukkan username dan password untuk mengakses dashboard.

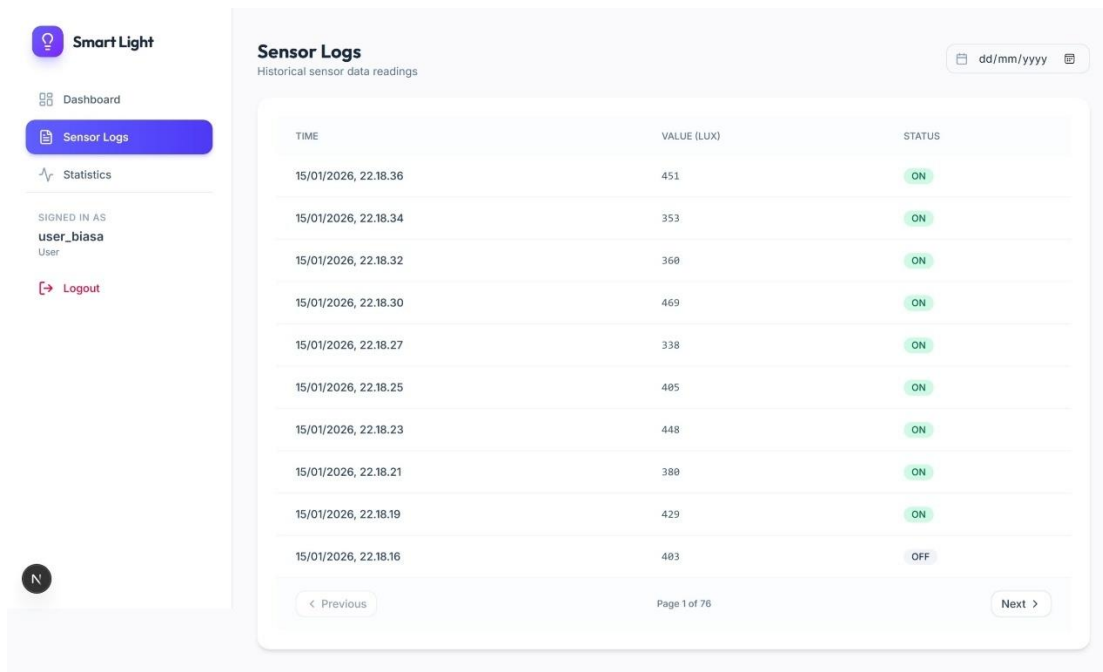
## 2. Dashboard User



Gambar 14 Dashboard User

Menampilkan dashboard user yang berisi status lampu real-time, nilai sensor cahaya terkini, dan konfigurasi sistem saat ini.

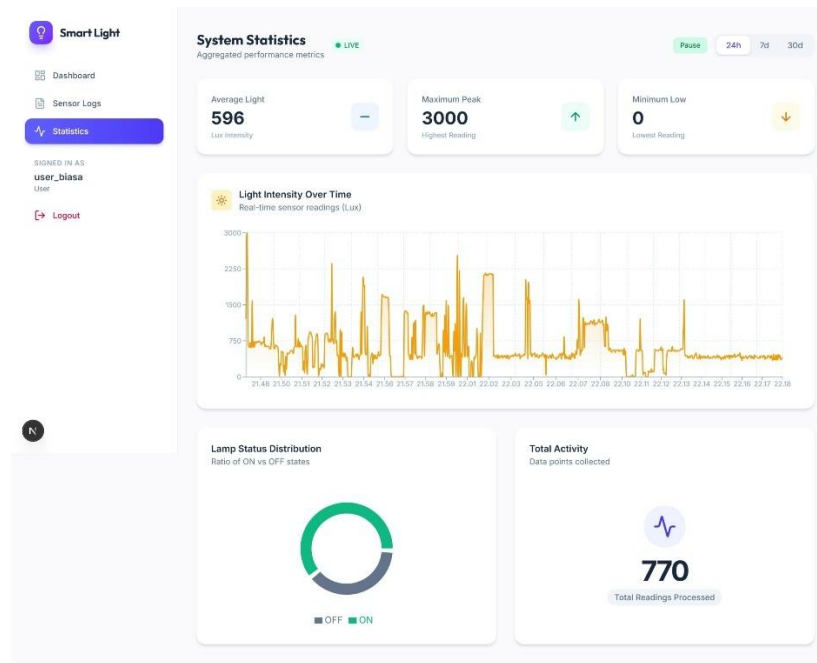
## 3. Halaman Sensor Logs



Gambar 15 Sensor Logs User

Menampilkan riwayat log sensor dalam bentuk tabel dengan informasi waktu, nilai cahaya, dan status lampu.

#### 4. Halaman Statistics



Gambar 16 Statistik User

Menampilkan grafik statistik penggunaan lampu dan nilai sensor cahaya dalam periode tertentu.

### 3.6.2 Tampilan Admin

#### 1. Halaman Login

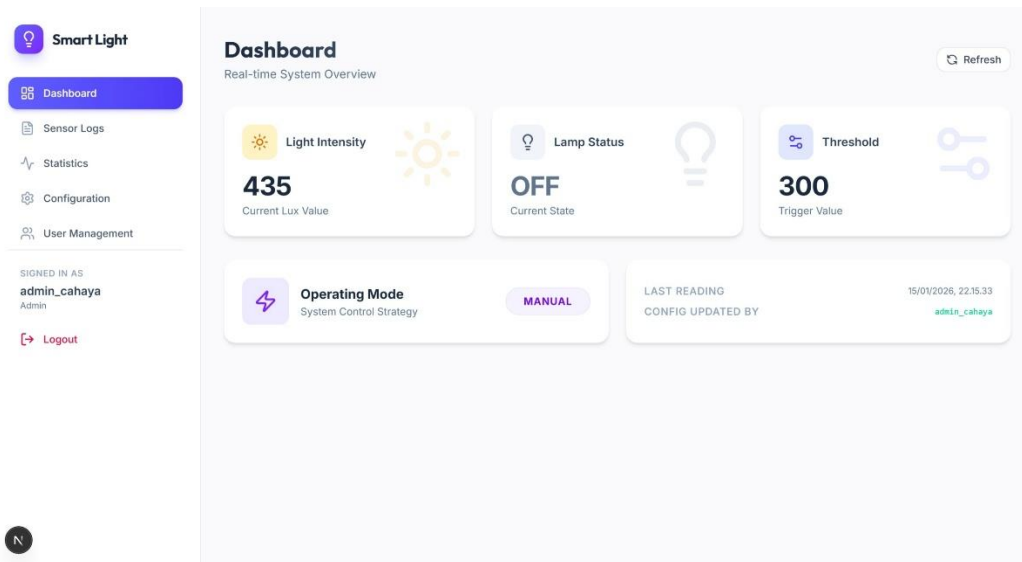
The screenshot shows the 'Welcome Back' login page for the smart light system. The page features a central white card with a purple lightbulb icon at the top. Below the icon, the text 'Welcome Back' is displayed, followed by the instruction 'Sign in to manage your smart home'. The login form includes two input fields: 'Username' with a placeholder 'Enter username' and 'Password' with a placeholder 'Enter password'. A purple 'Sign In' button is positioned below the password field. At the bottom of the card, a small text string reads 'Demo: admin\_cahaya / password123'. The background of the page is a soft gradient of purple and pink.

Gambar 17 Halaman Login



Menampilkan halaman login sistem. Admin memasukkan username dan password untuk mengakses dashboard.

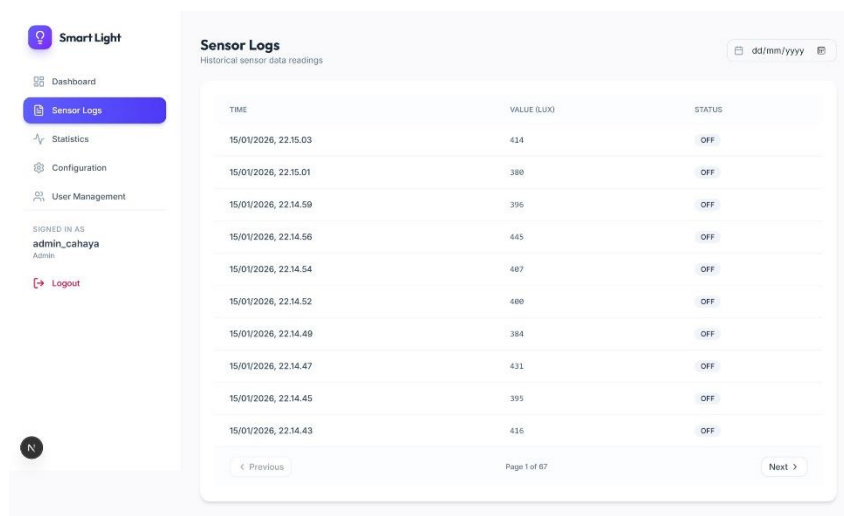
## 2. Dashboard Admin



Gambar 18 Dashboard Admin

Menampilkan dashboard admin dengan akses penuh ke semua fitur monitoring dan kontrol sistem.

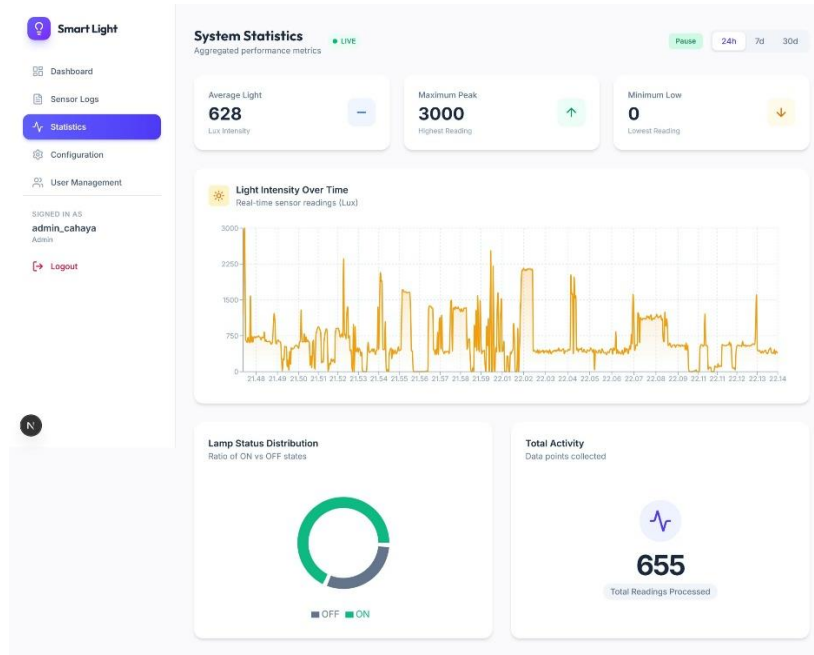
## 3. Sensor Logs Admin



Gambar 19 Sensor Logs Admin

Menampilkan halaman riwayat log sensor yang dapat diakses oleh admin.

#### 4. Statistics Admin



Gambar 20 Statistik Admin

Menampilkan halaman statistik dengan grafik penggunaan lampu dan data sensor.

#### 5. System Configuration

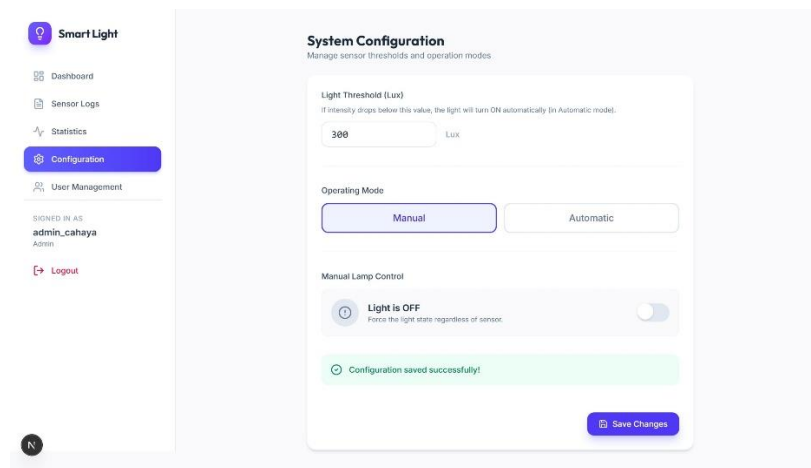
##### a. Light Threshold

The screenshot shows the 'System Configuration' page for 'Smart Light', specifically the 'Light Threshold' section. The left sidebar is identical to the previous screenshot, with 'Configuration' highlighted. The main content area has the title 'System Configuration' and subtitle 'Manage sensor thresholds and operation modes'. The 'Light Threshold (Lux)' section includes a text input field with the value '300' and a 'Lux' label, with a note: 'If intensity drops below this value, the light will turn ON automatically (in Automatic mode)'. The 'Operating Mode' section has two buttons: 'Manual' (selected) and 'Automatic'. The 'Manual Lamp Control' section features a toggle switch labeled 'Light is OFF' with the description 'Force the light state regardless of sensor.' and a 'Save Changes' button at the bottom right.

Gambar 21 Pengaturan Light Threshold

Menampilkan pengaturan Light Threshold (Lux) untuk menentukan batas nilai cahaya.

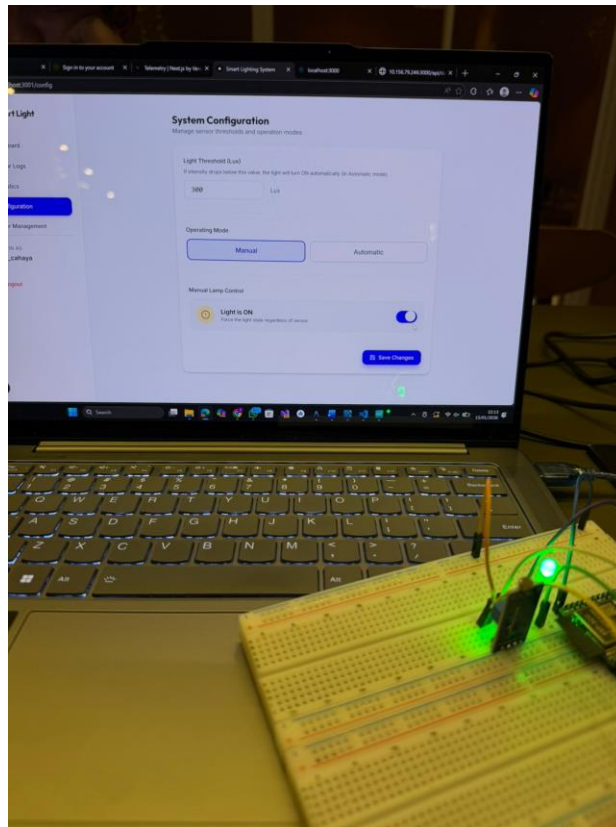
b. Operating Mode - Manual



Gambar 22 Operating Mode Manual

Menampilkan tampilan ketika Operating Mode diatur ke Manual.

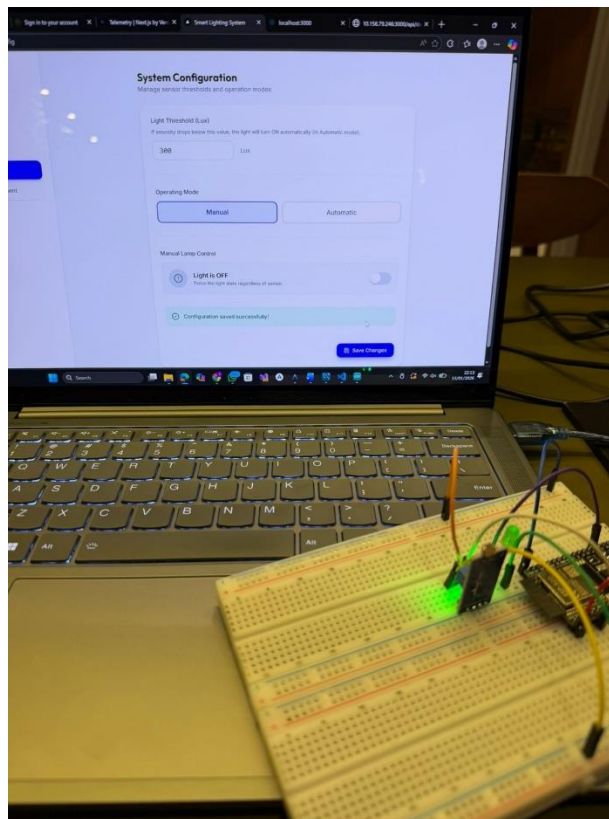
- **Manual Lamp ON**



Gambar 23 Manual Lamp ON

Menampilkan kondisi saat admin menyalakan lampu secara manual.

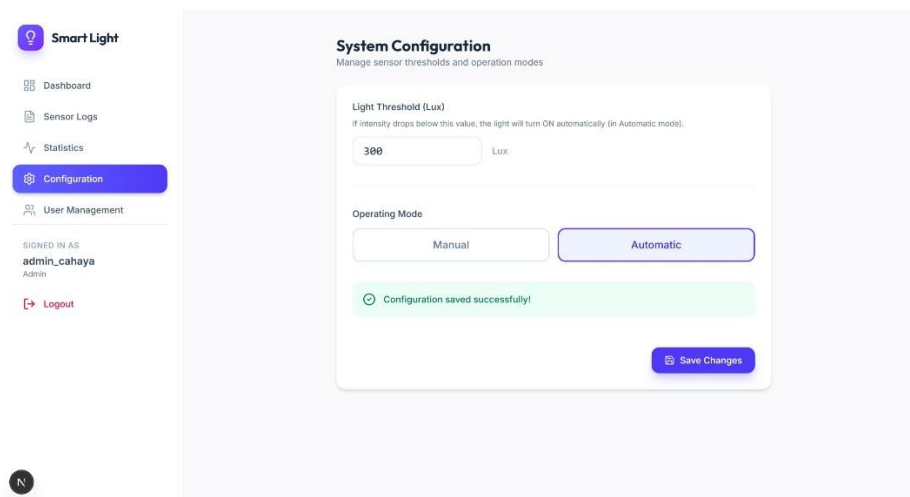
- **Manual Lamp OFF**



*Gambar 24 Manual Lamp OFF*

Menampilkan kondisi saat admin mematikan lampu secara manual.

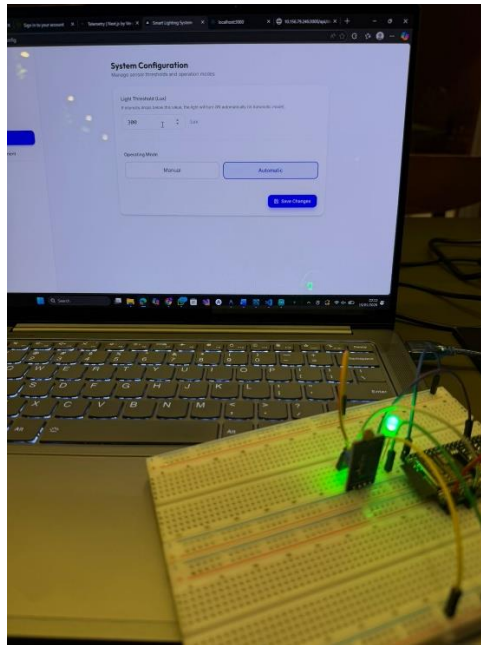
c. **Operating Mode - Automatic**



*Gambar 25 Operating Mode Automatic*

Menampilkan tampilan ketika Operating Mode diatur ke Automatic.

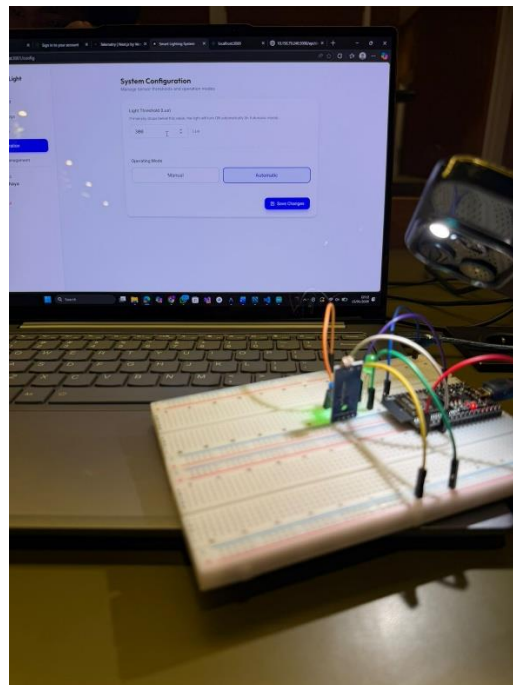
- **Auto Lamp ON**



*Gambar 26 Auto Lamp ON*

Menampilkan kondisi saat lampu menyala otomatis karena cahaya di bawah threshold.

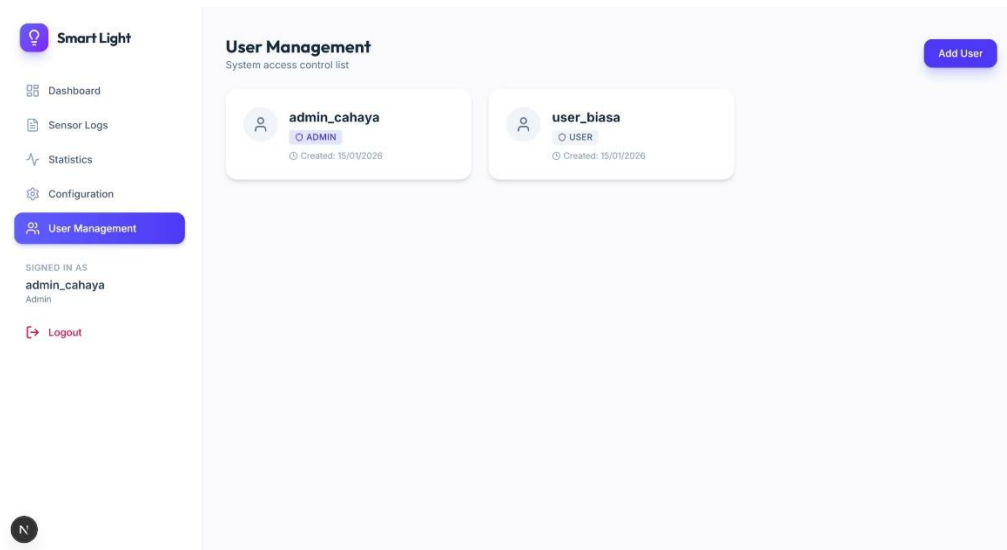
- **Auto Lamp OFF**



*Gambar 27 Auto Lamp OFF*

Menampilkan kondisi saat lampu mati otomatis karena cahaya di atas threshold.

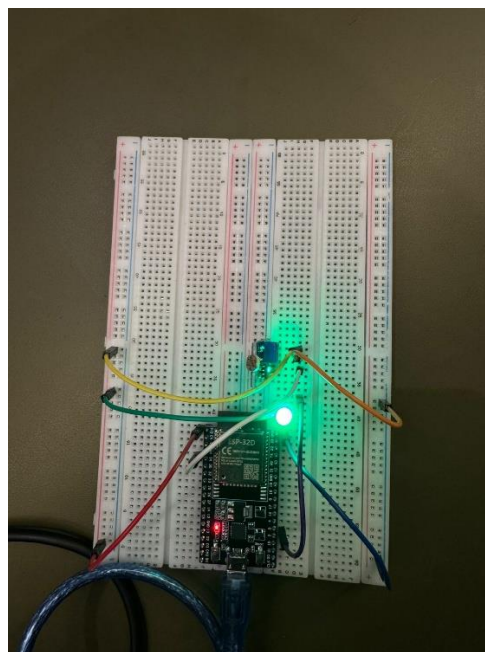
## 6. User Management



Gambar 28 Manage User

Menampilkan halaman manajemen user untuk menambah dan menghapus user.

### 3.6.3 Perangkat IoT (ESP32)



Gambar 29 Perangkat IOT

Menampilkan perangkat ESP32 yang terhubung dengan sensor LDR dan LED.

## **Bab 4 Penutup**

Sistem Smart Light IoT berhasil diimplementasikan dengan fitur:

1. Kontrol lampu otomatis berdasarkan sensor cahaya (LDR)
2. Kontrol lampu manual oleh admin
3. Monitoring real-time melalui dashboard web
4. Manajemen user (tambah/hapus)
5. Pencatatan log sensor dan statistik